Lab 3B: Subagents & Agent Teams

Duration: 35 min **After:** Subagents & Agent Teams lecture (Day 3, Block 2) **Prerequisite:** Lab 3A complete (MCP server configured).

> ***Optional Agent Teams setup:*** *Only if you plan to run the stretch, set* `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1` *in* `.claude/settings.json`*, then open a new Claude Code tab or instance. The core lab does not require this feature.*

Objective

Define restricted custom subagents, run a shallow nested delegation, apply a grader against a rubric, and complete a **parallel review sprint** that turns three independent agent findings into one implemented and independently verified improvement.

Deliverable (toolkit chain 10 of 12)

Two subagent definitions in `.claude/agents/`, a coordinator-produced quality report, and `labs/3b-agent-receipt.md` showing parallel findings, the selected fix, test evidence, and the grader's before-and-after verdict.

> ***START MARKER:*** *clean git status and a known-good baseline commit in* `business-dashboard`*; then open a new Claude Code tab or instance so project agent definitions and settings reload.*
>
> ***Recovery boundary:*** *checkpoints are useful for conversation and direct code edits, but do not assume they restore Bash side effects, background/subagent writes, or external changes. Git is the durable recovery layer. Commit before fan-out. Parallel writers need one task per worktree or explicit non-overlapping file ownership.*

Phase 1: Define Custom Subagents (7 min)

Subagents are markdown files in `.claude/agents/` — isolated instances with their own context, tools, and (optionally) model.

Step 1 — a restricted auditor:

```
Create a subagent at .claude/agents/security-auditor.md in the current project directory:

---
name: security-auditor
description: Audit code for security vulnerabilities. Use when asked to "check security", "audi
t for vulnerabilities", or "review auth code".
tools: Read, Grep, Glob
---

# Security Auditor
Check for: hardcoded secrets, SQL injection, unvalidated input, eval(), unsafe deserialization.
Output: CRITICAL / WARNING / INFO findings with file and line references.
```

Step 2 — a grader with a rubric:

```
Create a subagent at .claude/agents/grader.md:

---
name: grader
description: Grade completed work against a rubric. Use when asked to "grade", "score against
the rubric", or "run a review pass".
tools: Read, Grep, Glob
---

# Grader
Read the rubric file specified in the task. Score each criterion 0-2 (0=missing, 1=partial, 2=m
et).
Output a table: criterion | score | evidence (file:line). End with TOTAL: n/10 and a PASS
(>=8) or REWORK verdict. Never fix code — only grade it.
```

Step 3 — verify the definitions:

Confirm both files exist:

- `.claude/agents/security-auditor.md`
- `.claude/agents/grader.md`

Then ask Claude Code:

```
List the available project subagents. Confirm that security-auditor and grader are loaded, and
summarize each agent's tool restrictions in one sentence.
```

Expected output marker: Claude identifies both project subagents and reports that each is restricted to `Read`, `Grep`, and `Glob`. The `/agents` creation wizard has been removed in current Claude Code builds, so do not use `/agents` as the verification step. If Claude cannot find the agents, open a new Claude Code tab or instance so it reloads `.claude/agents/`, then run the verification prompt again.

Step 4 — first delegation:

```
Use the security-auditor subagent to audit everything in src/ and save findings to reports/
security.md
```

Expected output marker: the subagent runs in its own context, and `reports/security.md` appears with CRITICAL/WARNING/INFO findings. Current Claude Code backgrounds subagents by default. Follow its activity in the conversation, then use the returned report and created file as the evidence; no separate task command is required.

Phase 2: Nested Delegation — 2+ Levels (7 min)

Subagents can spawn subagents. The current default allows up to **three subagent layers below the main conversation** and can be changed with `CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH` (`1` disables nesting). You will run the shallow tree **you → coordinator → workers**.

```
Spawn a subagent acting as a coordinator with this task:
"Produce reports/quality-report.md for the src/ directory. Do NOT analyze the code yourself —
delegate: spawn one subagent to inventory all endpoints and their validation status, and a
second subagent to list every function missing error handling. Merge their two outputs into a
single report with sections: Endpoints, Error Handling Gaps, Top 3 Risks."
```

Watch the delegation tree: your session (level 0) → coordinator (level 1) → two workers (level 2).

Expected output marker: `reports/quality-report.md` exists with all three sections, and the transcript shows the coordinator spawning its own subagents — not doing the analysis itself.

```
Show me the delegation tree for that last task: who spawned whom, and what did each level contr
ibute?
```

> ***When to nest:*** *the coordinator holds the MERGE logic; workers hold the DETAIL. Nesting keeps your main context clean — you receive one report, not two raw dumps. Don't nest for simple sequential work; every level multiplies token cost.*

Phase 3: Grader-Style Review Pass (7 min)

The Performance Outcomes pattern: work is not "done" until a separate grader scores it against an explicit rubric.

Step 1 — write the rubric:

```
Create rubric.md with 5 criteria for src/routes/tasks.js:
1. All inputs validated with guard clauses
2. try/catch on every route
3. Parameterized SQL only
4. Error responses use { error: "message" }
5. No console.log left in production paths
```

Step 2 — grade:

```
Use the grader subagent to score src/routes/tasks.js against rubric.md
```

Expected output marker: a criterion/score/evidence table ending in `TOTAL: n/10` and PASS or REWORK.

Step 3 — close the loop. If REWORK:

```
Fix only the criteria the grader scored below 2, then have the grader re-score.
```

Expected output marker: second grading run scores higher. Save the final criterion/score/evidence table and verdict to `labs/3b-verdict.md`. Builder and grader are SEPARATE contexts — the grader can't be talked into leniency by the builder's reasoning. That separation is the point.

Phase 4: Agent Review Sprint (10 min)

Run a complete agent operating loop against the existing project: decompose the review, normalize the outputs, rank the evidence, implement one bounded fix, and verify it independently.

Step 1 — parallel read-only review:

```
Launch three read-only subagents in parallel:

1. API reviewer: inspect src/routes/tasks.js and src/routes/metrics.js for validation and
error-handling gaps.
2. Test reviewer: inspect test/tasks.test.js and test/metrics.test.js for missing coverage.
3. Dependency reviewer: inspect package.json and server.js for runtime or dependency risks.

Each must return the same table:
finding | file:line evidence | severity | minimal fix | test to prove it

Do not edit files.
```

Expected output marker: three independent reports using the same five-column contract. Reject and rerun any report that omits file evidence or the test that would prove the fix.

Step 2 — reconcile, implement one fix, and regrade:

```
Use the grader subagent to deduplicate the three reports, score the evidence, and select the
highest-value safe fix.

Implement only that fix in the main session. Run the targeted test, have the grader rescore the
result, and save the before-and-after evidence to labs/3b-agent-receipt.md.
```

If the reviewers find no valid code defect, select the strongest missing test case instead and add only that test.

Expected output marker: `labs/3b-agent-receipt.md` identifies all three reviewers, records the selected finding and evidence, lists the exact changed files and test command, and contains the grader's before-and-after verdict. Review the actual Source Control diff and test output before accepting the result.

> ***What this phase proves:*** *parallelism only helps when every worker receives a bounded scope and the same output contract. The grader supports the decision; the human still owns the diff and release decision.*

Optional Stretch: Agent Teams — EXPERIMENTAL (10 min)

Run this only after completing the core lab. Agent Teams may be unavailable or disabled by organization policy; it is not required for completion.

```
Create an agent team to add a /api/export/summary endpoint:
- architect: design the endpoint and data shape
- implementer: build it following CLAUDE.md
- reviewer: grade the result against rubric.md
Architect first, then implementer, then reviewer.
```

If the feature is unavailable, stop. Do not troubleshoot it during the lab. The stable core sprint above already demonstrates decomposition, isolated contexts, normalized results, grading, and human verification.

	Single session	Subagents (including nested)	Agent Teams
Coordination	you	main agent	team runtime and peer mailboxes
Context	one window	isolated per agent	isolated per teammate; shared checkout unless isolated
Cost	lowest	per-agent multiplier	highest
Use when	sequential work	bounded independent work	peers must negotiate during the task

---

FINISH MARKER — Success Checklist

- [ ] Two subagents defined in `.claude/agents/` with restricted tools
- [ ] Nested delegation ran: you → coordinator → 2 workers
- [ ] `reports/quality-report.md` was merged by the coordinator
- [ ] Grader scored work against `rubric.md` with evidence and a verdict
- [ ] Three parallel reviewers returned the same five-column contract
- [ ] Grader deduplicated and selected one evidence-backed improvement
- [ ] One bounded fix or missing test was implemented and tested
- [ ] `labs/3b-agent-receipt.md` records the before-and-after evidence
- [ ] Source Control diff, Problems panel, and targeted tests reviewed before the human decision
- [ ] Grader verdict treated as evidence, not a replacement for human review

If Stuck

Problem	Recovery
Subagent not found	Files must be in the project's `.claude/agents/`; open a new Claude Code tab or instance to reload them
Coordinator does the work itself	Re-prompt: "Do not analyze directly; delegate the two inspections and only merge their outputs"
Nested spawn refused	Check `CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH`; flatten to two read-only workers plus a main-session merge
Grader tries to fix code	Confirm its tools are restricted to Read, Grep, and Glob
Reviewer omits evidence	Reject that report and rerun only that reviewer with the five-column contract
Reviewers find no valid defect	Select the strongest missing test case and add only that test
Token usage grows quickly	Narrow each scope; every agent adds another context window

Debrief Questions

1. What did nesting keep out of the main conversation?
2. Why did every parallel reviewer need the same output contract?
3. What made the grader's selection auditable?
4. What evidence did you personally inspect before accepting the fix?